Tutor Agent Harness — Build Workflow

Oct 17, 2026 · @Roman Hunt

Overview

The harness is a single long-lived service, running as its own unprivileged Unix user, that turns each child message into at most three Claude calls: a cheap plan, one tutoring reply, and a cheap safety review. Everything else (retrieval, memory writes, cue detection) is deterministic code, not model tool use.

It splits into two layers:

- **Runtime** owns the process and anything with side effects: lifecycle, config, the Claude client, Oracle 26ai, the tool executor, safety gates, audit, and the turn loop that sequences them.
- **Context orchestration** owns what the model sees and which model sees it: reading the child's cues, planning the pedagogical move, choosing the model, assembling a token-budgeted prompt, and writing memory back.

The runtime never decides *what to say*; context orchestration never touches a socket or a cursor directly. That boundary is what lets you swap the model, the database, or the pedagogy independently.

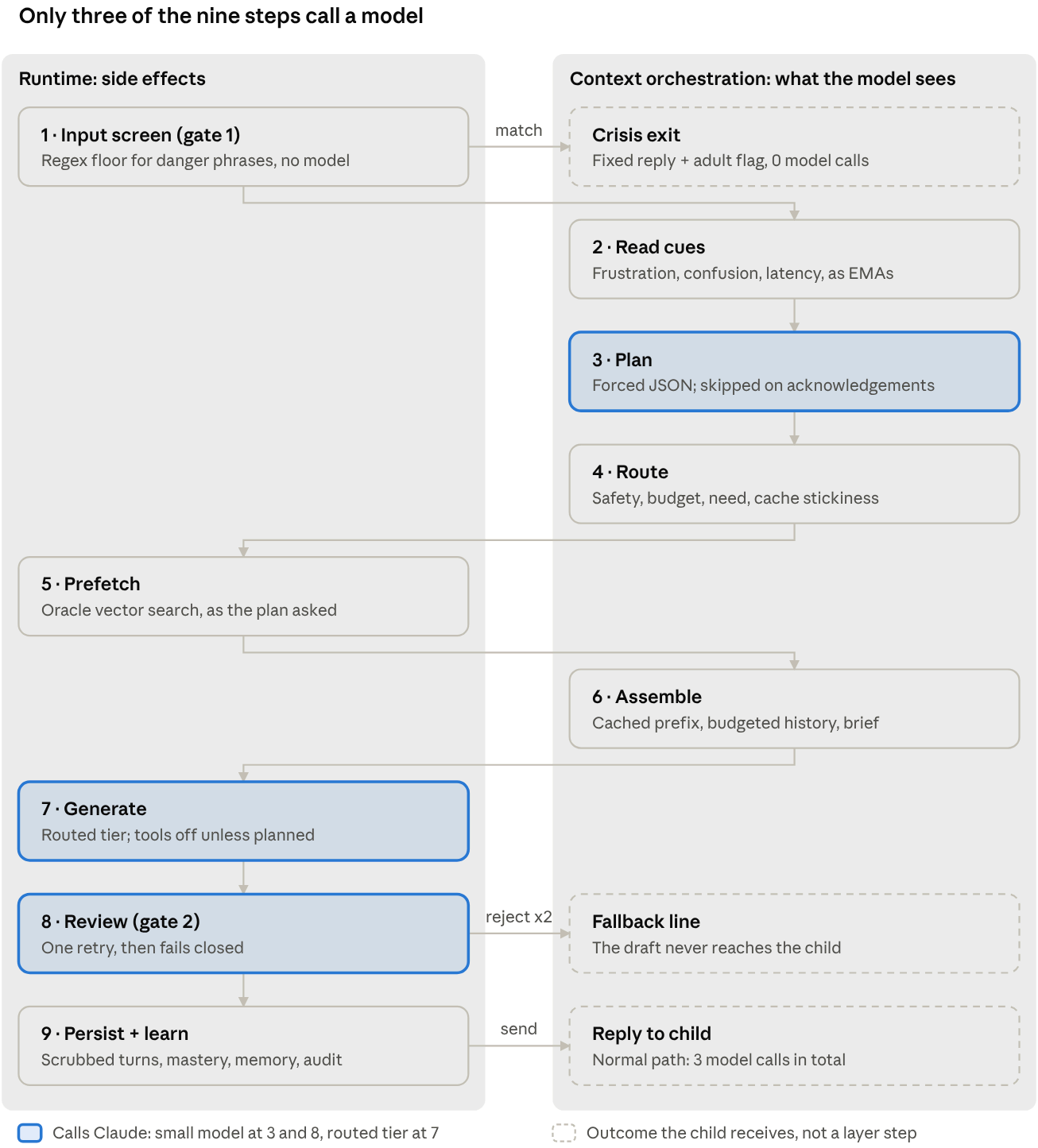
Assumptions I made (push back if wrong):

1. **Python 3.12.** The first-party Anthropic SDK and Oracle's `python-oracledb` thin driver are the most mature pair. You've said you don't read Python, so every file is commented heavily; the runtime layer is a reasonable Rust port later.
2. **Claude-only in front of the child.** The router only chooses among Claude tiers. The one non-Claude model is the ONNX embedding model running *inside* Oracle, which never generates text the child sees.
3. **Closed roster.** Teachers create students; there is no self-registration. The harness trusts a `learner_id` issued by your roster service.
4. **Hackathon scope.** One HTTP endpoint, no streaming, no UI. Those bolt on without touching either layer.

Design principles

- Prefer a fixed workflow over an open-ended agent when the task is predictable (Building effective agents).
- Plan once, then act once: the planner decides retrieval, tools and model tier up front, so the reply call rarely needs tools.
- Treat context as a budget, not a dump (Effective context engineering).
- Safety gates are synchronous and fail closed: nothing reaches the child unreviewed.
- Minimize data at write time; de-identify by design rather than by cleanup.

Architecture



per-turn flow · 9 steps across 2 layers

Each child message flows top to bottom. The runtime does all I/O; context orchestration decides what the model sees and which model sees it. Only steps 3, 7 and 8 call Claude, and step 3 is skipped for acknowledgements.

Setup, step by step

Six steps take a bare Linux host and an Oracle 26ai instance to a running, hardened tutor service. Each step ends with a check so you know it worked before moving on.

1. **Create the database users.** As a DBA in the PDB (`FREEPDB1` on 26ai Free), run `sql/00_users.sql`. It creates `TUTOR_APP` (what the harness uses) and `TUTOR_VAULT` (names, guardian contacts, consent), and gives the harness no grant on the vault.
   - Check: `SELECT username FROM all_users WHERE username LIKE 'TUTOR%';` returns both.
2. **Create the schema.** As `TUTOR_APP`, run `sql/01_schema.sql`: learner state (native `JSON`), turns, vector memory (`VECTOR(384, FLOAT32)`), curriculum chunks, usage, adult flags, audit log, and the `deidentify_learner` procedure.
   - Check: `SELECT table_name FROM user_tables;` lists seven tables.
3. **Load the embedding model into the database.** Download Oracle's pre-built augmented ONNX build of all-MiniLM-L12-v2, unzip it into the server-side `ONNX_DIR`, then run `sql/02_load_embedding_model.sql`. Oracle's docs describe exactly this flow: load the model with `DBMS_VECTOR.LOAD_ONNX_MODEL`, then embed in SQL with `VECTOR_EMBEDDING`.
   - Check: the script's smoke test returns `384`.
4. **Prepare the host.** From the repo root, `sudo ./deploy/setup_host.sh`. It creates a no-login `tutor` system user, installs code as root-owned and read-only to `tutor`, builds a venv, lays down `/etc/tutor-harness/env`, generates the pseudonym pepper, and installs the systemd unit.
   - Check: `id tutor` shows a system uid; `ls -l /etc/tutor-harness/creds` shows `0600` root files.
5. **Add secrets and enroll a demo learner.** Write your API key, DB password and bearer token into `/etc/tutor-harness/creds/`. Compute a demo pseudonym (README step 4) and insert one `learner_state` row.
   - Check: `systemd-analyze security tutor-harness` reports a low exposure score.
6. **Run and call it.** `systemctl enable --now tutor-harness`, then POST to `/v1/turn` (README step 6), or use `--repl` for a live demo. The response `meta` shows the path, tier, route reason and call count for each turn.
   - Check: a normal message reports `model_calls: 3`; "ok thanks" reports 2.

Tests without a network. `python3 -m unittest -v` runs 22 tests against in-memory fakes of Claude and Oracle, covering every turn path, the router, the assembler, cues and privacy. All pass. I could not reach PyPI from my sandbox, so the live SDK, driver and FastAPI paths are compiled but untested: run one real turn first thing after install.

Runtime layer

The runtime owns every side effect: the process, secrets, the Claude API, Oracle, tools, safety gates and the loop that sequences them. Each external system has exactly one module, so there is one place to audit, mock, or port.

Module	Owns	Key decision
`config.py`	Settings and secrets, loaded once	Secrets come from systemd credentials, not env vars
`llm.py`	The only Claude client	`structured_call()` gets JSON in one call by forcing a tool
`db.py`	The only Oracle client	Embeddings computed in-database; model name validated, never user input
`privacy.py`	Pseudonyms, PII scrub	HMAC with a pepper; scrub before every insert
`safety.py`	Input screen, output review	Deterministic floor first; review fails closed
`tools.py`	Two-tool allowlist + executor	Tool definitions never change, to protect the cache
`loop.py`	`TurnEngine`	Every call bound is enforced in code
`process.py`	HTTP service, REPL demo	Loopback bind; logs never contain child text

Process: its own user, nothing more

The agent runs as the `tutor` system user under `deploy/tutor-harness.service`. The unit makes the OS read-only (`ProtectSystem=strict`), drops every capability, blocks privilege gain (`NoNewPrivileges`), filters syscalls to `@system-service`, limits sockets to IP and Unix, and caps memory and tasks. Code is root-owned, so a compromised process can't rewrite its own prompts. Reference: systemd.exec.

For egress control, add an nftables rule matching `meta skuid tutor` that allows only the Oracle listener and the Claude API. That belongs in host config, not the unit.

Structured output in one round trip

The planner and reviewer must return JSON. Rather than parsing prose and retrying on bad JSON, `structured_call()` forces a tool whose input schema *is* the answer:

```
# runtime/llm.py
result = llm.complete(
    model=model, system=system, max_tokens=max_tokens,
    messages=[{"role": "user", "content": user_text}],
    tools=[tool_schema],
    tool_choice={"type": "tool", "name": tool_schema["name"]},  # model MUST
fill this schema
)
```

The "tool" is never executed; its input is the plan or review. See tool use.

Two safety gates

- **Gate 1, before any model call.** `screen_input()` matches a short list of unambiguous danger phrases (self-harm, being hurt, a stranger asking to meet or for photos, "keep it a secret"). A hit returns a fixed, advisor-reviewed reply, raises a `high` adult flag, and makes zero model calls.
- **Gate 2, after the draft.** One small-model call reviews the draft for safety, age fit, handing over graded answers, PII requests and human impersonation. A rejection allows one revised draft; a second rejection, or any reviewer error, sends a neutral fallback line. The unreviewed draft never reaches the child.

The same review call also returns mastery evidence and one memory note, so safety and learning bookkeeping share a round trip.

The turn engine

`TurnEngine.handle()` runs nine numbered steps; each comment in `loop.py` matches one. The call ceiling per path:

Path	Model calls
Crisis phrase	0

Path	Model calls
Acknowledgement ("ok thanks")	2: reply, review
Normal	3: plan, reply, review
Plan allowed tools and model used one	+1, then `tool_choice: none` forces text
Review rejected	+2 (revised reply and its review), then fallback

Context orchestration layer

This layer decides what the model sees and which model sees it. It never opens a socket or cursor; it calls the runtime's `LLM` and `Store` interfaces.

Cues: reading the child without a model call

`cues.py` scores each message for frustration, confusion and disengagement from word choice, capitals, punctuation, length and response latency. It also flags answer-seeking ("just tell me") and over-sharing (contact details). Scores feed an exponential moving average ($\alpha = 0.4$) stored per learner, so one outburst nudges the picture but a run of them changes it.

Confusion is tracked apart from frustration on purpose. Confusion that gets resolved tends to help learning; confusion that persists tends to slide into frustration or boredom (D'Mello & Graesser, 2012). So `confusion_streak` counts consecutive confused turns, and the planner only scaffolds once it reaches 2.

Text-only cues are crude next to multimodal affect sensing. They are cheap, deterministic and explainable to a teacher, which is the right trade for a first build.

Planner: one cheap call decides the turn

`planner.py` sends the small model a compact signal summary (not a transcript) and forces a `submit_plan` call returning:

- **move**: one of nine pedagogical moves (Socratic question, hint, worked example, check understanding, encourage, reframe simpler, suggest break, redirect, privacy reminder);
- **retrieval**: whether to fetch memories and lesson-plan excerpts, and with what query;
- **allow_tools**: normally false;
- **complexity** (1–5) and **risk** (none/low/high): inputs to the router.

The move rules in `prompts/planner.md` encode standard pedagogy: scaffold one step past independent ability (zone of proximal development), show worked examples to a stuck novice (Sweller, cognitive load), check understanding by explaining back, praise effort over ability (Mueller & Dweck, 1998).

A **fast path** skips the planner for 1–3 word acknowledgements, saving a call on a large share of children's turns.

Assembler: cache-ordered, budgeted context

`assembler.py` lays the prompt out so the expensive parts stay cached:

```
tools        TOOL_DEFS (constant)                  ┐
system       tutor_system.md + age-band rules      ┘ cache breakpoint 1 (stable
for days)
messages     prior turns                           ← cache breakpoint 2 (grows
each turn)
final user   <learner_brief> + <child_message>       never cached
```

Everything volatile (plan, signals, retrieved notes) rides in the final user message. Put it in the system prompt and it would invalidate the cached conversation behind it every turn. Child text is always wrapped in `<child_message>` tags, and the system prompt says those contents are words to respond to, never instructions.

Tokens are estimated at four characters each instead of calling the count-tokens endpoint, which would add a round trip per turn. The budget keeps 15% headroom and drops the oldest turns first. See prompt caching and effective context engineering.

Memory: recall before, keep after

- **Recall** happens only when the plan asks, using the plan's query, via in-database vector search. The tutor never "searches its memory" as a tool call.
- **Keep** takes the reviewer's single proposed note, scrubs it, caps it at 300 characters, and embeds it in Oracle.
- **Mastery** per skill updates as `p ← p + 0.2 × (outcome − p)`, enough to rank weak skills. Swap in Bayesian Knowledge Tracing once you have data to fit it.

Prompts as documented files

Each prompt in `context/prompts/` opens with an HTML comment stating its role, owner, and cache impact. The loader strips comments, so documentation costs no tokens. `tutor_system.md` carries the child-facing rules: AI disclosure, one question at a time, no handing over graded answers, no personal information, tell a trusted grown-up. Age rules cap replies at 40, 70 and 110 words for bands 6–7, 8–10 and 11–13, with the youngest band limited to the lesson plus stranger-danger basics.

Model selection and token economy

The router spends the cheapest Claude tier that is *safe* for this turn, and never economizes on a struggling or at-risk child. It lives in `context/router.py`, because which model sees the context is a context decision.

The ladder (all Claude, set in `.env`):

Tier	Default model	Used for
small	`claude-haiku-5-5`	Planner, reviewer, simple checks, budget guard
medium	`claude-sonnet-5-5`	Typical explanations
large	`claude-opus-5-5`	Risky moments, acute frustration, hard concepts for a struggling child

Cost weights in `.env` (1 / 3 / 5) are placeholders. Set them from the pricing page before drawing cost conclusions.

Rules, first match wins:

1. **Safety.** Planner risk `high`, or this-turn frustration ≥ 0.8 → large.
2. **Budget.** Learner at ≥ 90% of the daily token budget → small. Still Claude, still reviewed.
3. **Need.** Score = plan complexity, +1 for a worked example or reframe, +1 if the frustration trend > 0.5. Score ≤ 2 → small, ≤ 4 → medium, else large.
4. **Cache stickiness.** A downgrade waits while the prompt cache is warm, until 3 consecutive lower-need turns. Upgrades are never delayed.

Why rule 4 matters. Prompt caches belong to one model. A session's cached prefix on Sonnet, which is the system prompt plus the whole conversation so far, is worthless to Haiku. A switch re-bills that prefix at full price plus a cache write. A downgrade saves

money only if it sticks, and the saving grows with session length. Measure it rather than trusting the heuristic: every turn's `meta.calls` reports `cache_read` and `cache_write` per call.

Other token savers built in:

- The planner sees a signal summary, not the transcript, and is capped at 300 output tokens.
- The fast path drops the planner call on acknowledgements.
- Retrieval is prefetched, so the reply model needs no exploratory tool calls.
- Tool definitions are constant, keeping them inside the cached prefix.
- Token estimation is local (characters ÷ 4), avoiding a count-tokens round trip per turn.
- Daily usage weights cache reads at 0.1 of normal input, matching how they are billed (verify the ratio on the pricing page).

Planning first: how tool calls are eliminated

In the normal path the reply model makes zero tool calls, because everything a tool would have fetched is already in its context. An open-ended agent would instead discover what it needs one tool round at a time, paying a full round trip, and re-reading the whole context, for each.

Four mechanisms make that work:

1. **Decide before acting.** The planner names the retrieval queries, and the harness runs them as plain SQL before the reply call. This is the routing-then-workflow pattern from Building effective agents: a tutoring turn is predictable enough that a fixed sequence beats an autonomous loop.
2. **Define tools, disable them by default.** `TOOL_DEFS` is sent every time (stable for caching), but `tool_choice` is `{"type": "none"}` unless the plan sets `allow_tools`. The model can't call what it isn't allowed to call.
3. **Cap the rounds in code.** When tools are allowed, one round is the maximum: after it, `tool_choice` flips to `none` and the model must answer in text. The test `test_tool_round_is_capped_at_one` proves it.
4. **Fold side jobs into calls you already make.** Mastery evidence and memory notes come out of the safety review rather than separate extraction calls. Escalation doesn't depend on the tutor using a tool: the planner and reviewer can each raise an adult flag on their own.

The trade-off: when the planner misjudges retrieval, the reply is less informed than an agent that could go look. The `lookup_curriculum` tool covers that gap when the plan allows tools, and `tool_call` events in the audit log show how often it is needed.

Data stewardship and child safety

"De-identified, not deleted" is workable, but only if de-identification is a real, scheduled, irreversible step. Keeping identifiable children's data indefinitely is now explicitly off the table. I'm not a lawyer; treat this as engineering input for counsel, not legal advice.

What the rules require

- **COPPA (amended).** The FTC's amended rule took effect June 23, 2025, with compliance required by April 22, 2026. It requires a written, published data retention policy stating purposes, business need and a deletion timeframe, and it prohibits retaining children's personal information indefinitely (Davis Polk summary; rule text, 16 CFR 312).
- **FERPA.** Data that flows from schools is education records. Releasing it to researchers or the Department of Education needs FERPA-grade de-identification (34 CFR 99; Student Privacy Policy Office). Check Tennessee's student-privacy statutes too.
- **Anthropic's minors guidelines.** Products letting minors interact with Claude must add safeguards such as age verification, content moderation and monitoring. They must also comply with COPPA and disclose that the user is talking to an AI, and Anthropic audits compliance (Guidelines for Organizations Serving Minors).

How the harness implements it

Layer of protection	Where
Harness never sees names; HMAC pseudonyms only	`privacy.py`, `sql/00_users.sql` (vault schema, no grant)
Contact details scrubbed before any insert	`privacy.scrub()` in `loop._persist_turn`
Embeddings computed inside Oracle; child text never sent to an embedding API	`sql/02_*`, `db.py`
Logs carry events and ids, never child text	`process.py`, `audit_log`

Layer of protection	Where
Irreversible de-identification at retention end or on parent request	`deidentify_learner` in `sql/01_schema.sql`
AI disclosure, no personal-info requests, trusted-adult referral	`prompts/tutor_system.md`
Deterministic crisis screen + fail-closed review	`safety.py`
Teacher review queue before parents see notes	`adult_flags` table (product rule from your design)

Why de-identification drops text

Regex scrubbing catches emails, phones, URLs and street addresses. It misses names, schools, and "I live by the water tower." Free-text transcripts from children are the hardest data to de-identify, so `deidentify_learner` does not trust the scrubber. It:

1. NULLs all transcript text and deletes memory notes;
2. replaces the pseudonym everywhere with a random token and stores no mapping, so even the pepper can't relink it;
3. keeps the research-valuable residue: mastery and affect trajectories, moves, timings, model tiers, token counts, cohort attributes (class, age band).

That residue is what the Department of Education and researchers can actually use, and it's defensible under NIST SP 800-188. Two open items for your advisors: the retention window length, and whether `class_id` plus `age_band` is too fine-grained for small classes (k-anonymity).

Verification and next steps

All 22 unit tests pass against in-memory fakes of Claude and Oracle. The live SDK, driver and HTTP paths compile but have not run, because my sandbox couldn't reach PyPI. The first live turn is the real test.

What the tests prove:

Property	Test
Normal turn = exactly plan, reply, review	`test_normal_turn_is_exactly_three_calls_plan_reply_review`
"ok thanks" skips the planner	`test_acknowledgement_skips_the_planner`
Crisis phrase: zero model calls, high flag	`test_crisis_phrase_short_circuits_with_zero_model_calls_and_a_flag`
Two rejections or a reviewer outage → fallback, never the draft	`test_review_rejects_twice…`, `test_review_outage_fails_closed`
At most one tool round	`test_tool_round_is_capped_at_one`
No raw contact details or roster ids persisted	`test_raw_text_and_roster_id_are_never_persisted`
Router: safety > budget > need > stickiness	five `RouterTests`
Cache breakpoints placed; oldest history dropped first	`ContextTests`

Writing the tests caught one real bug: the acknowledgement pattern matched only single words, so "ok thanks" paid for a planner call. Fixed.

First live run, in order:

- [ ] Install, run the SQL smoke test, send one turn; confirm `meta.model_calls` is 3.
- [ ] Send a second turn within five minutes; confirm the reply call shows `cache_read` > 0. If it's 0, the system prompt may be below the model's minimum cacheable length: add pedagogy content, which costs nothing once cached.
- [ ] Confirm `{"type": "none"}` as `tool_choice` is accepted by your SDK version.
- [ ] Ask Anthropic for the child-safety system prompt their guidelines mention, and merge it into `tutor_system.md`.
- [ ] Have your child-development advisors review the fixed crisis replies and the age-band word limits.

After the hackathon: streaming replies; a teacher dashboard over `adult_flags`; syllabus upload and chunking; session summaries as memory; Bayesian Knowledge Tracing; an nftables egress allowlist for the `tutor` uid; a Rust port of the runtime layer.

References

Pages marked † were opened during this work; the rest are standard references cited from knowledge, worth a click before you quote them to judges.

Agent and context design

- Building effective agents, Anthropic: workflows vs. agents, routing
- Effective context engineering for AI agents, Anthropic
- Prompt caching, Claude docs: breakpoints, TTL, per-model caches
- Tool use overview, Claude docs: `tool_choice`, forced tools
- Pricing, Claude docs: set the router's cost weights from here

Oracle Database 26ai

- † SQL quick start: embedding model uploaded into the database, Oracle docs
- † Pre-built ONNX model for in-database embeddings, Oracle blog
- † ONNX embedding workflows in Oracle AI Database 26ai with Python, Oracle blog
- † AI Vector Search in 23ai/26ai, Oracle-Base
- python-oracledb documentation

Child safety, privacy, compliance

- † Guidelines for Organizations Serving Minors, Anthropic
- † Child safety guidance for developers, Anthropic
- † FTC prioritizes COPPA enforcement as new obligations take effect, Davis Polk
- † Updated COPPA rule requirements take effect April 22, The Toy Association
- COPPA Rule, 16 CFR Part 312, eCFR
- FERPA, 34 CFR Part 99, eCFR; Student Privacy Policy Office
- NIST SP 800-188, De-Identifying Government Datasets

Learning science

- D'Mello & Graesser (2012), Dynamics of affective states during complex learning, *Learning and Instruction*
- Sweller (1988), Cognitive load during problem solving, *Cognitive Science*
- Mueller & Dweck (1998), Praise for intelligence can undermine children's motivation, *JPSP*
- Corbett & Anderson (1994), Knowledge tracing, *User Modeling and User-Adapted Interaction*

- Vygotsky, zone of proximal development (overview)

Host hardening

- systemd.exec and systemd credentials